



华南理工大学
South China University of Technology

本科生课程报告

代码覆盖率及路径分析检测工具

学 院	计算机科学与工程学院
--------	------------

专 业	本硕强基专业型（本科）
--------	-------------

学 生 姓 名	徐 宇 鸿
---------	-------

学 生 学 号	202267330269
---------	--------------

指 导 教 师	李 方
---------	-----

提 交 日 期	2026 年 4 月 21 日
---------	-----------------

目 录

目 录	II
第一章 课题背景与实验目标.....	1
1.1 课题背景	1
1.2 作业目标	1
1.3 工具方案概述	1
1.4 PageIndex 实验对象说明	2
第二章 系统设计与实现	3
2.1 设计目标与总体流程	3
2.2 模块划分与调用关系	3
2.3 数据模型设计	4
2.4 运行时追踪器实现	6
2.5 静态分支提取实现	9
2.6 覆盖率分析与报告生成	11
2.7 命令行接口与执行流程	14
2.8 与 PageIndex 的接入方式	15
2.9 本章小结	16
第三章 实验过程与结果分析	17
3.1 实验设计与评价指标	17
3.2 实验环境与实验对象	17
3.2.1 实验环境	17
3.2.2 被测对象说明	18
3.3 测试用例设计	18
3.3.1 设计原则	18
3.3.2 测试用例定义	19
3.4 实验执行过程	19
3.4.1 执行命令	19
3.4.2 执行步骤	20
3.5 实验结果	20

3.5.1	总体覆盖率结果	20
3.5.2	分支交集与并集分析	21
3.5.3	累计覆盖结果	21
3.6	结果解释与路径分析	22
3.6.1	TC1: 普通模式基线路径	22
3.6.2	TC2: 流式混合消息路径	22
3.6.3	TC3: 列表型消息内容路径	22
3.6.4	实验设计说明	23
3.7	实验结论与局限	23
3.7.1	实验结论	23
3.7.2	仍然存在的局限	23
3.8	本章小结	24
第四章	使用文档	25
4.1	使用方式概览	25
4.2	命令行接口	25
4.2.1	基本语法	25
4.2.2	参数说明	25
4.2.3	常见命令	25
4.3	Python API 使用	26
4.3.1	最小调用流程	26
4.3.2	本项目中的实际调用方式	27
4.3.3	函数调用记录	28
4.4	报告格式说明	28
4.4.1	文本报告	28
4.4.2	JSON 报告	29
4.4.3	HTML 报告	30
4.5	支持范围与结果解释	30
4.5.1	当前版本支持的控制结构	30
4.5.2	覆盖率指标的含义	30
4.5.3	追踪边界	31
4.6	本项目中的推荐使用流程	31
4.7	本章小结	32

附录 A 核心源代码	33
A.1 数据模型 (models.py).....	33
A.2 运行时追踪器 (tracer.py)	34
A.3 控制流图构建器 (cfg_builder.py)	38
A.4 报告生成器 (reporter.py).....	40
A.5 命令行接口 (cli.py).....	44

第一章 课题背景与实验目标

1.1 课题背景

代码覆盖率和执行路径分析是软件测试中的基础能力。前者回答“哪些代码被跑到了”，后者回答“程序是怎么走到那里的”。在课堂作业场景里，重点不只是拿到一个覆盖率百分比，而是把追踪、分支识别和结果呈现这几件事自己实现一遍，弄清楚工具背后的工作机制。

Python 生态里已经有成熟的覆盖率工具，但这次作业更强调原理验证和过程可解释性。因此，本文的核心工作不是介绍某个业务项目，而是设计并实现一个面向 Python 程序的覆盖率与路径分析工具，再用真实项目代码来检验它是否能给出有用的测试信息。

1.2 作业目标

根据课程要求，本次作业需要完成以下几项任务：

- 记录 Python 程序运行时的执行路径；
- 分析顺序、选择、循环等基本控制结构；
- 统计代码覆盖情况，并生成可读的执行分支报告；
- 提供清晰的代码说明和使用文档，保证工具具备基本的可维护性。

围绕这些要求，本文希望回答三个具体问题：第一，能否从静态源代码中提取出需要关注的控制结构；第二，能否在程序运行时稳定记录实际执行路径；第三，能否把两类信息汇总成对测试设计有帮助的报告。

1.3 工具方案概述

本文实现的工具命名为 **PathTracer**。它采用“静态分析 + 动态追踪”的组合方案：先用 **ast** 模块扫描目标文件，找出其中的 **if**、**for**、**while**、**except** 等控制结构；再用 **sys.settrace** 在程序运行时记录实际执行到的代码行；最后将静态分支信息与动态执行结果对齐，输出覆盖率统计和执行路径报告。

这里需要说明的是，当前版本统计的是“控制结构命中情况”。也就是说，只要某个控制结构所在的起始行在运行时被执行到，就认为该结构被命中。它还不能像成熟工具那样严格区分 **if** 的真分支和假分支，也不对所有可能路径做穷举。但对本次课程作业来说，这个定义已经足够支撑路径记录、结构识别和报告生成三项核心要求。

1.4 PageIndex 实验对象说明

PageIndex 是一个面向长文档处理的索引与问答项目，能够把 PDF、Markdown 等文档整理成层次化结构，并在此基础上完成检索与回答生成。本文并不把 **PageIndex** 作为研究主体，而是把它当作真实项目样本，用来验证 **PathTracer** 在实际代码上的覆盖率统计与路径分析能力。换句话说，**PageIndex** 在本文里的角色是“被测程序”，不是“被介绍的系统”。

从项目结构上看，与本次实验最相关的部分主要有三层：

- 索引构建模块：`pageindex/page_index.py`，负责解析源文档、抽取层次结构并生成树形索引结果；
- 检索与回答模块：`pageindex/search.py`，负责列出已建索引文档、筛选相关文档、并发搜索节点、拼接上下文并生成回答；
- 命令行入口脚本：`agent_pageindex.py`，负责接收用户问题、创建 **Agent**，并调用 `search_pageindex` 工具完成一次完整问答流程。

本文将 `agent_pageindex.py` 选为主要实验对象，原因有三点。第一，它处在 **PageIndex** 的入口位置，既能代表真实使用方式，又能把下层检索流程连接进来，因此比单独写一个玩具示例更有说服力。第二，这个脚本的代码规模适中，适合在课程报告里做较细致的路径分析，不至于因为项目过大而让实验描述失去重点。第三，它本身已经包含了多种典型控制结构：`_extract_text` 中有针对 **None**、字符串、列表等不同输入类型的条件分流；`_print_verbose_stream` 中有多层 **for** 循环、消息去重以及工具消息和模型消息的分类处理；`main` 函数中则包含参数解析、环境变量检查、普通模式与 **verbose** 模式的路径分叉。

如果进一步看调用链，`agent_pageindex.py` 并不是一个孤立的小脚本。它在创建 **Agent** 后，会把用户问题交给 `search_pageindex`；而该工具内部又会继续执行文档目录读取、文档筛选、节点搜索、上下文构造和答案生成等步骤。也就是说，虽然本报告直接统计的目标文件是入口脚本，但这个入口脚本背后连接的是完整的 **PageIndex** 检索流程，因此实验结果既保留了真实项目的复杂性，又把分析范围控制在课程作业可讨论的规模内。

基于上述考虑，本文对 **PageIndex** 的介绍只保留与实验设计相关的部分：它是什么、入口脚本做了什么、为什么适合拿来做覆盖率分析。至于文档问答系统本身的业务细节，并不是本次报告要展开的重点。

第二章 系统设计与实现

2.1 设计目标与总体流程

PathTracer 的设计目标很明确：给定一个 Python 源文件，先找出其中有哪些需要统计的控制结构，再在程序运行时记录实际执行路径，最后把两类信息合并成覆盖率报告。为了把这个过程讲清楚，本章不只介绍模块划分，还直接结合核心代码说明系统是怎样工作的。

从执行顺序看，整个系统可以拆成四步。第一步，命令行接口读取用户参数，确定目标文件、输出格式和传给目标程序的参数。第二步，静态分析器扫描目标源文件，抽取 `if`、`for`、`while` 和 `except` 等控制结构。第三步，运行时追踪器利用 `sys.settrace` 记录目标文件中实际执行过的代码行，并顺带构造函数调用树。第四步，报告生成器把“静态上有哪些结构”和“动态上执行了哪些位置”对齐，输出文本、JSON 或 HTML 报告。

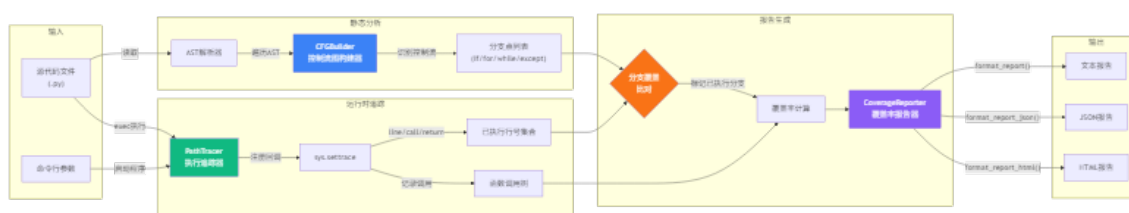


图 2-1 PathTracer 系统架构图

这个流程看起来不复杂，但它有一个好处：每个模块都只做一件事。静态分析器不关心程序实际怎么运行，追踪器不负责判断代码里到底有多少个控制结构，报告生成器只负责把前面两部分的结果汇总。这样代码更容易写，也更容易解释。

2.2 模块划分与调用关系

PathTracer 的源码集中在 `pathtracer` 目录下，核心模块有五个：

- `models.py`：定义统一的数据结构，负责在模块之间传递分支、执行路径和报告数据；
- `tracer.py`：实现运行时追踪，记录执行行号和函数调用信息；
- `cfg_builder.py`：实现静态分析，从抽象语法树中提取控制结构；
- `reporter.py`：把静态结果和动态结果合并成覆盖率报告；
- `cli.py`：提供命令行入口，负责调度前面四个模块。

它们之间的调用关系也很直接。命令行入口先创建 `PathTracer` 实例，再执行目标程序；程序运行结束后，`CoverageReporter` 调用 `CFGBuilder` 获取静态分支集合，同时从追踪器中取出执行行集合；最后将命中结果写回 `CoverageReport`。也就是说，`models.py` 定义的是公共数据格式，剩下几个模块围绕这些数据结构顺次工作。

2.3 数据模型设计

系统之所以能保持清晰，关键在于先把数据结构定下来。下面这段代码来自 `path tracer/models.py`，它定义了本工具最核心的几个对象：

Listing 2.1 PathTracer 的核心数据模型

```
from dataclasses import dataclass, field
from typing import Set, Dict, List, Optional
from enum import Enum
```

```
class BranchType(Enum):
    # 统一定义可统计的控制结构类型
    SEQUENTIAL = "sequential"
    IF = "if"
    ELSE = "else"
    ELIF = "elif"
    FOR = "for"
    WHILE = "while"
    TRY = "try"
    EXCEPT = "except"
```

```
@dataclass
class CodeBranch:
    # 表示源码中的一个静态控制结构入口
    file_path: str
    line_no: int
    branch_type: BranchType
    end_line: int = 0
    is_executed: bool = False
```



```
@dataclass
class ExecutionPath:
    # 保存一次运行中某个文件的动态执行结果
    file_path: str
    executed_lines: Set[int] = field(default_factory=set)
    executed_branches: Dict[int, CodeBranch] = field(
        default_factory=dict
    )
    function_calls: List["FunctionCall"] = field(
        default_factory=list
    )
```

```
@dataclass
class CoverageReport:
    # 汇总最终统计结果，供文本/JSON/HTML 三种报告复用
    total_branches: int = 0
    executed_branches: int = 0
    branches_by_type: Dict[BranchType, int] = field(
        default_factory=dict
    )
    coverage_percentage: float = 0.0
    file_reports: Dict[str, ExecutionPath] = field(
        default_factory=dict
    )
```

这组模型的分工很明确。**BranchType** 统一描述控制结构类型，**CodeBranch** 表示源代码中的静态分支点，**ExecutionPath** 记录某个文件在一次运行中的动态执行结果，**CoverageReport** 则负责汇总最终统计结果。这样设计的直接好处是，各模块之间不需要靠零散字典传值，而是始终围绕同一套结构工作。

这里还有一个值得指出的地方：**BranchType** 的定义其实比当前实现更宽。枚举里已经预留了 **SEQUENTIAL**、**ELSE**、**ELIF** 和 **TRY** 等类型，但本版本的静态提取器还没有把这些类型全部识别出来。这说明数据模型一开始就给后续扩展留了空间，而具体识别能力还可以继续补齐。

其中 **CodeBranch** 和 **ExecutionPath** 是最关键的一对。前者描述“代码里本来

有哪些结构”，后者描述“这次运行到底走了哪些位置”。报告生成器后面要做的事情，本质上就是把这两份信息对齐。

2.4 运行时追踪器实现

PathTracer 是系统的动态部分。它借助 Python 标准库里的 `sys.settrace` 注册回调函数，在解释器执行目标程序时实时接收事件。核心实现如下：

Listing 2.2 运行时追踪器的核心回调逻辑

```
class PathTracer:
    def __init__(self):
        # 每个目标文件对应一份独立的执行记录
        self.execution_paths = {}
        self._original_trace = None
        self._target_file = None
        self.function_calls = []
        self._call_stack = []

    def trace_file(self, file_path: str) -> "PathTracer":
        # 指定本次要追踪的目标文件，并预先创建容器
        self._target_file = file_path
        self.execution_paths[file_path] = ExecutionPath(
            file_path=file_path
        )
        return self

    def _trace_callback(self, frame, event: str, arg):
        if not self._target_file:
            return None

        code = frame.f_code
        # 只保留目标文件中的事件，过滤掉标准库和第三方库
        if not code.co_filename.endswith(self._target_file):
            return self._trace_callback

        if event == "line":
            # line 事件只记录“哪一行被执行到了”
            line_no = frame.f_lineno
```

```

        path = self.execution_paths[self._target_file]
        path.executed_lines.add(line_no)

    elif event == "call":
        # call 事件额外记录函数名、参数和调用层级
        func_name = code.co_name
        line_no = frame.f_lineno
        args = {}
        arg_count = code.co_argcount
        arg_names = code.co_varnames[:arg_count]
        for arg_name in arg_names:
            if arg_name in frame.f_locals:
                args[arg_name] = repr(frame.f_locals[arg_name])

        call_depth = len(self._call_stack)
        func_call = FunctionCall(
            function_name=func_name,
            file_path=code.co_filename,
            line_no=line_no,
            call_depth=call_depth,
            arguments=args,
        )

        if self._call_stack:
            # 若当前已有父调用，则把它挂到父节点下面
            self._call_stack[-1].children.append(func_call)
        else:
            # 否则它就是一棵调用树的根节点
            self.function_calls.append(func_call)
            path = self.execution_paths[self._target_file]
            path.function_calls.append(func_call)
            self._call_stack.append(func_call)

    elif event == "return":
        # return 事件与最近一次 call 配对，用于补全返回值
        if self._call_stack:
            func_call = self._call_stack.pop()

```

```

        if arg is not None:
            func_call.return_value = repr(arg)

    return self._trace_callback

```

这段代码里有几个实现细节值得单独说明。

第一，`trace_file()` 并不直接启动追踪。它先保存目标文件路径，再初始化对应的 `ExecutionPath`。这样后面的回调函数就能把结果直接写入固定位置。

第二，`code.co_filename.endswith(self._target_file)` 这一句限制了追踪范围。它只记录目标文件中的事件，不把标准库、第三方库甚至 `PathTracer` 自己的内部调用算进去。对于课程作业来说，这个过滤非常重要，否则输出会被大量无关代码淹没。

第三，`line` 事件和 `call/return` 事件被分开处理。前者只负责记录哪些行被执行过；后者负责构造函数调用树。这样做的结果是，覆盖率统计和调用树展示虽然都来自同一个回调，但在数据层面是分开的，不会互相干扰。

第四，函数调用树依靠 `_call_stack` 维护层级关系。每次发生 `call` 事件时，系统就把新的 `FunctionCall` 压栈；发生 `return` 事件时，再把对应调用弹出并补上返回值。这个栈结构是整棵调用树能够成立的关键。

为了便于使用，追踪器还实现了上下文管理器接口：

Listing 2.3 追踪器的启停方式

```

def __enter__(self):
    # 保存原来的追踪状态，避免影响外部环境
    self._original_trace = sys.gettrace()
    sys.settrace(self._trace_callback)
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    # 退出时恢复原来的追踪函数
    sys.settrace(self._original_trace)
    return False

```

这里的思路很朴素：进入 `with` 块时注册新的追踪函数，退出时恢复原来的追踪状态。这样 `PathTracer` 的启停是成对出现的，外部调用也更简洁。

2.5 静态分支提取实现

动态追踪只能告诉我们“程序执行到了哪里”，但不能告诉我们“代码里一共有多少个控制结构”。这部分工作由 **CFGBuilder** 完成。它没有真的去构建一张复杂的控制流图，而是利用 **AST** 扫描出需要统计的控制结构入口：

Listing 2.4 基于 AST 的静态分支提取

```
class CFGBuilder:
    def build_from_file(self, file_path: str):
        # 先读源码，再交给 ast 构建语法树
        with open(file_path, "r", encoding="utf-8") as f:
            source = f.read()

            tree = ast.parse(source, filename=file_path)
            self.branches = []

            for node in ast.walk(tree):
                # 逐个检查 AST 节点，筛出需要统计的控制结构
                branch = self._extract_branch(file_path, node)
                if branch:
                    self.branches.append(branch)

            return self.branches

    def _extract_branch(self, file_path: str, node: ast.AST):
        if isinstance(node, ast.If):
            # 记录 if 结构入口及其代码范围
            return CodeBranch(
                file_path=file_path,
                line_no=node.lineno,
                branch_type=BranchType.IF,
                end_line=self._get_end_line(node),
            )
        elif isinstance(node, ast.For):
            # 记录 for 循环入口
            return CodeBranch(
                file_path=file_path,
                line_no=node.lineno,
```

```

        branch_type=BranchType.FOR,
        end_line=self._get_end_line(node),
    )
elif isinstance(node, ast.While):
    # 记录 while 循环入口
    return CodeBranch(
        file_path=file_path,
        line_no=node.lineno,
        branch_type=BranchType.WHILE,
        end_line=self._get_end_line(node),
    )
elif isinstance(node, ast.ExceptHandler):
    # 记录 except 处理分支
    return CodeBranch(
        file_path=file_path,
        line_no=node.lineno,
        branch_type=BranchType.EXCEPT,
        end_line=self._get_end_line(node),
    )

return None

```

这一部分实现比运行时追踪简单，但同样关键。它的核心做法是：先用 `ast.parse` 把源码转成抽象语法树，再用 `ast.walk` 遍历所有节点，只保留与控制结构相关的几类节点。每发现一个目标节点，就创建一个 `CodeBranch` 对象。

这里有两个设计取舍需要讲清楚。第一，当前工具只统计 `if`、`for`、`while` 和 `except`，并没有覆盖 Python 里所有可能的控制结构，例如 `with`、`match-case` 或异步语法。第二，它统计的是“结构入口”，而不是“结构方向”。例如 `if` 语句会被记成一个分支点，但不会再细分真分支和假分支。这也是为什么本文更准确的说法是“控制结构命中率”。

另一个不太显眼但很有用的细节，是为每个结构记录结束行号：

Listing 2.5 控制结构结束行号的计算

```

def _get_end_line(self, node: ast.AST) -> int:
    # Python 3.8+ 优先直接使用 AST 节点的结束行号
    if hasattr(node, "end_lineno") and node.end_lineno is not None:
        return node.end_lineno

```

```

# 若没有 end_lineno, 就递归取 body 中最靠后的行号
if hasattr(node, "body") and node.body:
    return max(self._get_end_line(n) for n in node.body)
# 再退一步, 就把起始行号当作结束行号
return getattr(node, "lineno", 0)

```

这个字段对当前的命中判定并不是必需的, 但它对报告展示很有帮助。尤其是在 HTML 或 JSON 报告里, 知道一个控制结构覆盖了哪一段代码, 比只知道起始行号更直观。

2.6 覆盖率分析与报告生成

静态分支和动态执行路径准备好之后, CoverageReporter 开始真正计算覆盖率。核心分析逻辑如下:

Listing 2.6 覆盖率统计的核心实现

```

class CoverageReporter:
    def __init__(self):
        self.cfg_builder = CFGBuilder()

    def analyze(self, file_path: str, tracer: PathTracer) -> CoverageReport:
        # 静态部分: 先找出该文件里所有可统计的控制结构
        all_branches = self.cfg_builder.build_from_file(file_path)
        # 动态部分: 再取出本次运行真正命中的代码行
        executed_lines = tracer.get_executed_lines(file_path)

        for branch in all_branches:
            # 若控制结构起始行被执行到, 则认为该结构被命中
            if branch.line_no in executed_lines:
                branch.is_executed = True

        total = len(all_branches)
        executed = sum(1 for b in all_branches if b.is_executed)

        by_type = {}
        for branch_type in BranchType:
            # 统计每类控制结构在源码中一共出现了多少次

```

```

        type_branches = [
            b for b in all_branches if b.branch_type == branch_type
        ]
        by_type[branch_type] = len(type_branches)

    return CoverageReport(
        total_branches=total,
        executed_branches=executed,
        branches_by_type=by_type,
        coverage_percentage=(
            (executed / total * 100) if total > 0 else 0.0
        ),
        file_reports={
            file_path: ExecutionPath(
                file_path=file_path,
                executed_lines=executed_lines,
                # 只把已命中的结构写进最终报告
                executed_branches={
                    b.line_no: b for b in all_branches if b.
is_executed
                },
            )
        },
    )

```

这个方法的判断规则很直接。如果某个控制结构的入口行号出现在执行行集合里，就把它标记为已执行。源码里对应的判断条件，就是 `branch.line_noinexecuted_lines`。这个判定规则并不追求复杂，但足够稳定，也和前面静态模块的“入口统计”口径保持一致。

完成统计之后，报告生成器会继续把结果转成不同格式。文本输出适合直接放进终端或论文正文；JSON 输出适合实验结果留档和后处理；HTML 输出则更偏可视化展示。下面这段代码展示了 JSON 报告的组织方式：

Listing 2.7 JSON 报告的组织方式

```

def format_report_json(self, report: CoverageReport) -> str:
    # 先组织顶层统计字段
    data = {

```



```

        "coverage_percentage": report.coverage_percentage,
        "total_branches": report.total_branches,
        "executed_branches": report.executed_branches,
        "branches_by_type": {
            bt.value: count for bt, count in report.branches_by_type.
items()
        },
        "file_reports": {},
        "function_calls": [],
    }

    for file_path, exec_path in report.file_reports.items():
        # 再逐文件补充执行行和已命中的结构列表
        file_data = {
            "executed_lines": sorted(
                list(exec_path.executed_lines)
            ),
            "executed_branches": [],
        }
        for _, branch in sorted(exec_path.executed_branches.items()):
            file_data["executed_branches"].append(
                {
                    "line_no": branch.line_no,
                    "branch_type": branch.branch_type.value,
                    "end_line": branch.end_line,
                }
            )
        data["file_reports"][file_path] = file_data

    # 最后序列化为 JSON 字符串
    return json.dumps(data, indent=2)

```

从这个结构可以看出，报告生成器不只是输出一个覆盖率百分比。它把后续分析需要的信息也保留下来了，包括执行行集合、已命中分支列表和函数调用树。这也是第三章能够直接引用结果文件展开分析的原因。

不过从源码细节看，当前实现还有一个没有完全打通的地方。**PathTracer** 在运行时已经记录了 **function_calls**，但 **analyze()** 在重建 **ExecutionPath** 时只写

入了执行行和已命中分支，并没有把函数调用树一并带入报告对象。这意味着 JSON 和 HTML 侧虽然预留了函数调用树的序列化与展示逻辑，但在当前版本里，这部分能力还没有真正贯通到最终报告里。把这一点写出来，比简单地说“系统支持调用树”更准确。

2.7 命令行接口与执行流程

如果没有命令行入口，前面几个模块只能分散地手动调用。PathTracer 的 `cli.py` 做的事情，就是把“解析参数、运行目标程序、触发分析、输出报告”这一整套流程串起来。核心代码如下：

Listing 2.8 命令行入口的执行流程

```
def main():
    # 原始参数既包含 PathTracer 自己的选项，也包含目标程序参数
    raw_argv = sys.argv[1:]
    cli_argv = []
    program_args = []

    index = 0
    while index < len(raw_argv):
        arg = raw_argv[index]
        if arg in ("--output", "-o", "--format", "-f"):
            # 这些参数属于追踪器自身
            cli_argv.extend([arg, raw_argv[index + 1]])
            index += 2
            continue
        if arg in ("--quiet", "-q"):
            cli_argv.append(arg)
            index += 1
            continue
        # 第一个非追踪器参数视为目标脚本，剩余参数转交给它
        cli_argv.append(arg)
        program_args = raw_argv[index + 1:]
        break

    args = parser.parse_args(cli_argv)
    tracer = PathTracer().trace_file(args.program)
    reporter = CoverageReporter()
```

```

try:
    # 重写 sys.argv, 让目标脚本像平常一样读取命令行参数
    sys.argv = [args.program] + program_args
    with tracer:
        with open(args.program) as f:
            # 在当前进程里执行目标脚本, 才能被 sys.settrace 捕获
            code = compile(f.read(), args.program, "exec")
            exec(
                code,
                {"__name__": "__main__", "__file__": args.program},
            )
finally:
    # 目标脚本结束后立即生成报告
    report = reporter.analyze(args.program, tracer)

```

这段代码里最值得解释的是两点。

第一, 命令行参数被分成了两部分。一部分属于 PathTracer 自己, 例如输出格式、输出文件路径和静默模式; 另一部分属于被测程序本身。这里通过手动扫描 `raw_argv`, 把两类参数拆开处理, 避免追踪器自己的参数被错误传给目标脚本。

第二, 目标程序不是通过导入模块调用, 而是通过 `compile + exec` 的方式在当前进程里执行。只有目标脚本和追踪器处在同一个 Python 进程里, `sys.settrace` 才能真正捕获到执行事件。对一个路径分析工具来说, 这个实现细节比命令行参数本身更重要。

2.8 与 PageIndex 的接入方式

有了前面的命令行入口, PathTracer 接入 PageIndex 就变得很简单。实验时只需要把 `agent_pageindex.py` 作为目标文件, 再把查询参数追加到命令后面即可。例如:

Listing 2.9 PathTracer 接入 PageIndex 的方式

```

python -m pathtracer.cli -f json -o docs/coverage_tc2.json \
    agent_pageindex.py -- --query "test" --verbose

```

这里 PathTracer 并不需要修改 PageIndex 的业务代码, 也不需要给入口脚本额外打补丁。它只是在目标脚本运行前挂上追踪回调, 在脚本结束后做一次静态分析和结果汇总。这样的接入方式有两个好处: 一是侵入性很低, 不会影响被测项目原本的执行逻辑; 二是实验结果更接近真实使用场景, 而不是为了测试工具专门写一个配套程序。

2.9 本章小结

本章从源码角度展开说明了 **PathTracer** 的实现。系统先通过 `models.py` 统一数据结构,再由 `tracer.py` 记录动态执行路径,由 `cfg_builder.py` 提取静态控制结构,由 `reporter.py` 计算覆盖率并生成多格式报告,最后通过 `cli.py` 把整个流程组织成一个可直接运行的工具。这样写下来可以看出, **PathTracer** 的实现并不依赖复杂框架,主要依赖的是对 **Python** 自身运行机制和抽象语法树的合理利用。这也是它能够较容易接入 **PageIndex** 并完成真实项目实验的原因。

第三章 实验过程与结果分析

3.1 实验设计与评价指标

为保证实验结果具有可重复性，本章采用受控实验方式对 `agent_pageindex.py` 进行覆盖率分析。被测对象仍然是 `PageIndex` 的真实入口脚本，但实验时不直接依赖外部网络调用，而是通过桩替换的方式固定 `Agent` 和工具返回值。这样处理的目的，是把实验变量收紧，让每一组用例都对应一个明确的控制流目标。

本章实验主要回答三个问题：

- `PathTracer` 能否稳定识别 `agent_pageindex.py` 中全部 22 个控制结构入口；
- 在普通模式、流式模式和列表型消息输入三种场景下，脚本分别会走到哪些路径；
- 三组测试用例的结果并集能否覆盖全部 22 个控制结构入口。

本章继续使用第二章定义过的控制结构命中率作为核心指标：

$$\text{Coverage} = \frac{\text{ExecutedBranches}}{\text{TotalBranches}} \times 100\%$$

这个公式里的分子表示“控制结构起始行被执行到”的数量，分母表示静态分析阶段识别出的控制结构总数。这个口径依然不是严格意义上的真假分支覆盖，因此后文即使出现 100% 覆盖率，也只能说明 22 个控制结构入口都被执行到了，不能说明每个判断的真分支和假分支都被分别验证过。

3.2 实验环境与实验对象

3.2.1 实验环境

实验在当前项目环境中完成，关键配置如表 3-1 所示。

表 3-1 实验环境配置

项目	配置
运行平台	Darwin arm64
Python 版本	3.13.11
主要依赖	langchain>=1,<2, langchain-openai, python-dotenv==1.1.0
被测脚本	<code>agent_pageindex.py</code>
追踪工具	自研工具 <code>PathTracer</code> (包名 <code>pathtracer</code>)
实验驱动脚本	<code>tests/run_agent_pageindex_experiments.py</code>
结果文件	<code>docs/coverage_tc1.json</code> 、 <code>docs/coverage_tc2.json</code> 、 <code>docs/coverage_tc3.json</code>

其中, `tests/run_agent_pageindex_experiments.py` 是本章实验的驱动脚本。它不会修改 `agent_pageindex.py` 本身, 而是在实验进程里临时安装假的 `langchain.dotenv` 和 `pageindex.search` 模块, 再通过 `runpy.run_path(..., run_name="__main__")` 的方式直接执行真实脚本。这样既保住了“被测对象是 `PageIndex` 入口脚本”这一前提, 又把外部返回值变成了可精确控制的输入。

3.2.2 被测对象说明

`agent_pageindex.py` 的总代码量约 130 行, 控制结构主要集中在三个函数中:

- `_extract_text`: 负责统一处理 `None`、字符串、列表等不同类型的消息内容;
- `_print_verbose_stream`: 负责消费流式消息, 内部包含多层 `for` 循环、重复消息过滤、工具消息和模型消息分流;
- `main`: 负责参数解析、环境变量检查, 以及普通模式和 `verbose` 模式之间的主流程切换。

静态分析结果表明, 该脚本共包含 22 个可统计的控制结构入口, 其中 `if` 结构 17 个, `for` 结构 5 个。按函数拆分后的分布如表 3-2 所示。

表 3-2 `agent_pageindex.py` 静态控制结构分布

函数	if 入口数	for 入口数	合计
<code>_extract_text</code>	5	1	6
<code>_print_verbose_stream</code>	7	4	11
<code>main</code>	5	0	5
总计	17	5	22

这个分布说明, `_print_verbose_stream` 和 `_extract_text` 两部分集中了脚本中大多数控制结构。如果消息形态不可控, 这两部分就很容易留下明显覆盖空洞。

3.3 测试用例设计

3.3.1 设计原则

本章测试用例围绕目标路径进行设计。每组用例只负责覆盖一类关键控制结构, 三组结果求并集后再评估 22 个控制结构入口的整体覆盖情况。

设计原则如下:

- 被测对象固定为 `agent_pageindex.py`, 不换成 `demo` 脚本;
- 外部依赖全部桩化, 避免网络和 LLM 输出随机性影响结果;

- 每组用例只改变一种关键消息形态，便于解释覆盖率增量来自哪里；
- 所有结果统一输出为 JSON，方便直接做行号集合分析。

3.3.2 测试用例定义

三组实验场景如表 3-3 所示。

表 3-3 实验测试用例

用例	场景名	设计目的
TC1	normal_string	普通模式下返回字符串型最终答案，作为普通调用路径基线
TC2	verbose_mixed	流式模式下返回混合消息，覆盖非字典块、重复消息、工具调用、空消息、ToolMessage 和 AIMessage 路径
TC3	normal_list	普通模式下返回列表型消息内容，专门覆盖 _extract_text 的列表输入路径

这里最关键的是 TC3。该用例专门针对 _extract_text 的列表输入路径设计，使最终消息内容固定为：

Listing 3.1 TC3 中固定的列表型消息内容

```
[ "alpha fragment", { "type": "text", "text": "beta fragment" } ]
```

这样一来，_extract_text 中“判断为列表”“遍历列表”“字符串元素处理”“字典元素处理”四个控制结构都能稳定触发。

3.4 实验执行过程

3.4.1 执行命令

三组实验通过同一个驱动脚本执行，对应命令如下：

Listing 3.2 实验执行命令

```
python3 tests/run_agent_pageindex_experiments.py \
    --scenario normal_string --output docs/coverage_tc1.json

python3 tests/run_agent_pageindex_experiments.py \
    --scenario verbose_mixed --output docs/coverage_tc2.json

python3 tests/run_agent_pageindex_experiments.py \
    --scenario normal_list --output docs/coverage_tc3.json
```

3.4.2 执行步骤

每组实验都按相同步骤完成：

- 1) 在实验进程中安装假的 `langchain`、`dotenv` 和 `pageindex.search` 模块；
- 2) 根据场景名创建对应的假 Agent；
- 3) 用 PathTracer API 将追踪目标固定为 `agent_pageindex.py`；
- 4) 通过 `runpy.run_path` 以 `__main__` 方式执行目标脚本；
- 5) 用 `CoverageReporter` 对执行结果做静态-动态对齐，并输出 JSON 报告。

这里值得强调一点：实验虽然用了驱动脚本，但真正被追踪的文件始终是 `agent_pageindex.py`，而不是驱动脚本本身。因此，报告中的行号、分支数和覆盖率都仍然属于 `PageIndex` 入口脚本。

3.5 实验结果

3.5.1 总体覆盖率结果

三组实验结果如表 3-4 所示。

表 3-4 PageIndex 实验覆盖率结果

测试用例	已命中分支数	总分支数	覆盖率	执行行数
TC1	7	22	31.8%	55
TC2	16	22	72.7%	81
TC3	11	22	50.0%	62

TC1 仍然是最低的，只覆盖了普通模式下的基础路径：参数解析、环境检查、`agent.invoke` 返回结果读取，以及 `_extract_text` 的字符串输入处理。它的作用更多是作为基线，而不是承担补覆盖的任务。

TC2 是增量最大的用例，覆盖率达到 72.7%。这一组新增的 16 个命中分支由事先设计好的混合流式消息稳定触发。具体来说，这一组消息同时覆盖了：

- `chunk` 不是字典时的过滤；
- `payload` 不是字典时的过滤；
- 重复消息的去重；
- 工具调用消息与工具调用列表遍历；
- 空内容消息的跳过；
- `ToolMessage` 和 `AIMessage` 两类消息的分流打印。

TC3 的覆盖率是 50.0%，虽然低于 TC2，但它补的是 `_extract_text` 的四个关键列表路径分支。对整个实验设计来说，TC3 的价值不在于单次覆盖率高，而在于它把普通字符串输入难以覆盖的列表路径单独固定下来，形成可重复命中的测试对象。

3.5.2 分支交集与并集分析

为了更清楚地看到每组用例各自贡献了什么，把三组结果的分支集合拆开后可得到表 3-5。

表 3-5 三组测试用例的分支集合关系

类别	行号	对应逻辑
三组共有	25, 27, 88, 116, 129	文本提取入口、字符串判定、API Key 检查、 <code>verbose</code> 判定和脚本入口判断
仅 TC2 命中	43, 44, 46, 47, 50, 52, 57, 58, 64, 67, 73	流式块遍历、非法块过滤、消息循环、去重、工具调用、空消息处理、工具消息和模型消息分流
仅 TC3 命中	29, 31, 32, 34	<code>_extract_text</code> 的列表判定、列表遍历、字符串元素处理和字典元素处理
仅 TC1 命中	无	基线用例本身不提供独占增量

从这个结果可以看出，TC1 主要提供普通模式基线，本身不带来独占增量；决定整体覆盖效果的是 TC2 和 TC3。TC2 负责覆盖 `_print_verbose_stream` 中的流式处理逻辑，TC3 负责覆盖 `_extract_text` 的列表路径，两者分工清晰，也符合“围绕控制结构设计实验”的思路。

3.5.3 累计覆盖结果

把三组实验的已命中分支做并集统计之后，可以得到完整的 22 个控制结构入口：

Listing 3.3 三组实验结果的分支并集

```
25, 27, 29, 31, 32, 34, 43, 44, 46, 47, 50, 52,
57, 58, 64, 67, 73, 88, 116, 121, 123, 129
```

这意味着三组实验结果求并集后，累计覆盖率达到：

$$\frac{22}{22} \times 100\% = 100.0\%$$

这里的“100%”只对应控制结构入口的全覆盖。它说明 `agent_pageindex.py` 中所有 22 个静态控制结构都至少被执行到过一次；但它不意味着真假分支方向都被完整区分，也不意味着底层 `PageIndex` 检索模块已经被完整测透。这一点后面还会再说明。

3.6 结果解释与路径分析

3.6.1 TC1：普通模式基线路径

TC1 的路径最简单。程序进入 `main` 后，完成参数解析、环境变量检查和 `Agent` 创建，随后走普通模式的 `agent.invoke` 分支，在第 121 行和第 123 行判断返回结果是否为字典、是否包含消息列表，最后把最后一条消息内容送进 `_extract_text`。由于返回内容为普通字符串，所以 `_extract_text` 只会停留在第 25 行和第 27 行附近，不会进入列表路径。

这个用例的意义在于给后两组实验提供对照。它告诉我们：如果只验证最常见的普通调用路径，入口脚本里其实还有一大半结构不会被触发。

3.6.2 TC2：流式混合消息路径

TC2 是本组实验的主力用例。它把程序显式带进 `verbose` 模式，并且在流式返回里依次制造了五类事件：

- 非字典 `chunk`，触发第 44 行过滤；
- 字典 `chunk` 但 `payload` 不是字典，触发第 47 行过滤；
- 带 `tool_calls` 的消息，触发第 57 行和第 58 行；
- 重复消息，触发第 52 行；
- 空内容消息、`ToolMessage` 和 `AIMessage`，分别触发第 64、67、73 行。

这种设计比单纯加一个 `--verbose` 参数更有效，因为它不是笼统地进入流式模式，而是把流式模式内部的关键分支一个个拆开打。这样得到的 72.7% 覆盖率，不依赖真实消息流的随机形态，而是直接由测试数据控制。

3.6.3 TC3：列表型消息内容路径

TC3 是本章实验中用于补齐列表处理路径的关键用例。它依然走普通模式，但把最终消息内容固定成“字符串 + 文本字典”的列表。这样程序在调用 `_extract_text` 时，会连续经过以下结构：

- 第 29 行：判断 `payload` 是 `list`；
- 第 31 行：遍历列表元素；
- 第 32 行：处理字符串元素；
- 第 34 行：处理 `\{"type":"text"\}` 形式的字典元素。

这四处结构都集中在同一条列表输入路径上。TC3 把这条路径单独抽出来验证，所以它虽然单次覆盖率只有 50.0%，但在整个实验设计里反而是不可替代的一组。

3.6.4 实验设计说明

本组实验之所以采用受控输入而不是直接依赖真实查询返回，主要有两个原因。

第一，变量更可控。查询参数、外部返回形态和脚本内部路径这三者如果同时变化，覆盖率差异就很难解释清楚；将外部返回固定为预设场景后，每个增量都可以明确对应到具体控制结构。

第二，结论更可复查。三组结果都可以通过 `tests/run_agent_pageindex_experiments.py` 重新生成，不依赖网络、不依赖模型当时的随机输出，也不依赖某次数据库返回恰好是什么内容。对于课程实验报告来说，这一点非常重要，因为它让“为什么打到了这些行”变成了可验证的事实，而不是一次偶然运行的现象。

3.7 实验结论与局限

3.7.1 实验结论

本章实验表明，PathTracer 可以在 PageIndex 真实入口脚本 `agent_pageindex.py` 上稳定工作，并通过受控输入把 22 个控制结构入口全部命中。具体来说：

- TC1 覆盖普通模式基线路径，覆盖率为 31.8%；
- TC2 覆盖流式处理路径，覆盖率为 72.7%；
- TC3 覆盖列表型消息内容路径，覆盖率为 50.0%；
- 三组结果取并集后，累计覆盖率达到 100.0%。

这说明 PathTracer 至少在入口脚本这一层，已经具备了较完整的控制结构检测能力。更重要的是，这一覆盖结果来自明确的测试目标与受控输入设计，而不是依赖外部返回内容的偶然形态。

3.7.2 仍然存在的局限

即使本章已经拿到了 100.0% 的控制结构命中率，也不能把这个结果说得过头。它至少还有两个边界。

- 第一，这个 100.0% 只代表控制结构入口全覆盖，不代表真假分支方向全覆盖。第 88 行和第 116 行就是最典型的例子：这一行判断被执行到了，但条件真假并没有被分别统计。

- 第二，本章重点覆盖的是 `agent_pageindex.py` 入口脚本，而不是 `pageindex/search.py` 这类更深层模块。也就是说，当前实验主要验证了“入口脚本路径是否可测”，还没有展开到底层检索逻辑的更深层覆盖分析。

这两个限制并不削弱本章的主要结论，但需要如实写出来。否则只报一个 100%，很容易让人误以为整个 `PageIndex` 项目都已经被充分测试了。

3.8 本章小结

本章围绕 `PageIndex` 入口脚本 `agent_pageindex.py` 设计了一组可重复的受控实验方案。该方案保持被测对象不变，但把外部依赖全部桩化，并通过 `tests/run_agent_pageindex_experiments.py` 生成可复现的覆盖率结果。三组用例分别覆盖普通模式基线路径、流式混合消息路径和列表型消息内容路径，累计覆盖率达到 100.0%。

更关键的是，本章把“覆盖率为什么会变化”讲清楚了：TC2 负责打通流式处理逻辑，TC3 负责补齐 `_extract_text` 的列表路径，二者合起来才真正把 22 个控制结构入口全部覆盖。这样得到的实验结果更适合作为正式实验章节的依据。

第四章 使用文档

4.1 使用方式概览

PathTracer 提供命令行接口和 Python API 两种使用方式。命令行接口适合直接追踪单个 Python 脚本并生成覆盖率报告；Python API 适合嵌入测试代码、实验驱动脚本或其他自动化流程中。两种方式最终都围绕两个核心类展开：PathTracer 负责记录运行时执行行，CoverageReporter 负责将动态执行结果与静态控制结构对齐并生成报告。

在本项目中，如果只是想快速查看某个脚本的大致覆盖情况，直接使用命令行接口即可；如果要像第三章那样为 `agent_pageindex.py` 注入受控输入、替换外部依赖并批量生成 JSON 结果，则更适合使用 Python API。

4.2 命令行接口

4.2.1 基本语法

命令行入口位于 `pathtracer/cli.py`，可通过模块方式直接调用：

Listing 4.1 PathTracer 命令行基本语法

```
python -m pathtracer.cli [--output OUTPUT] [--quiet] \  
    [--format {text,html,json}] program [program_args ...]
```

其中，`program` 表示被追踪的 Python 文件，后面的 `program\_args` 会原样传递给目标程序。如果目标程序本身也带有命令行参数，建议显式写出 `--` 分隔符，这样更不容易和 PathTracer 自己的参数混淆。

4.2.2 参数说明

表 4-1 给出了当前命令行接口支持的主要参数。

4.2.3 常见命令

下面给出三种最常见的命令行用法。

Listing 4.2 命令行接口常见用法

```
# 1. 直接在终端输出文本报告  
python -m pathtracer.cli target.py
```

表 4-1 PathTracer 命令行参数说明

参数	说明
<code>program</code>	需要追踪的 Python 脚本路径
<code>-f, --format</code>	报告输出格式, 可选 <code>text</code> 、 <code>html</code> 、 <code>json</code> , 默认值为 <code>text</code>
<code>-o, --output</code>	报告输出文件路径; 若省略该参数, <code>text</code> 格式直接输出到终端, <code>html</code> 和 <code>json</code> 会分别默认写入 <code>coverage-report.html</code> 与 <code>coverage-report.json</code>
<code>-q, --quiet</code>	静默模式, 只隐藏目标程序的标准输出, 不影响最终报告输出
<code>program_args</code>	传递给目标程序的参数列表; 若以 <code>--</code> 开头, PathTracer 会自动去掉这个分隔符

2. 生成 JSON 报告

```
python -m pathtracer.cli -f json -o report.json \
    target.py -- arg1 arg2
```

3. 追踪 agent_pageindex.py, 并把脚本参数放在 -- 之后

```
python -m pathtracer.cli -f html -q \
    agent_pageindex.py -- --query "test" --verbose
```

其中第三条命令对应的是本项目里的真实入口脚本 `agent_pageindex.py`。需要注意, 这种直接追踪方式依赖目标脚本本身的运行环境已经准备好, 例如相关依赖包已经安装、环境变量 `DEEPSEEK_API_KEY` 已经设置等。若要获得可重复的课程实验结果, 更推荐使用第三章中的受控驱动脚本。

4.3 Python API 使用

4.3.1 最小调用流程

当命令行接口无法满足需求时, 可以直接调用 Python API。最小调用流程如代码 4.3 所示。

Listing 4.3 PathTracer 最小 API 调用流程

```
from pathtracer import CoverageReporter, PathTracer

tracer = PathTracer().trace_file("target.py")
reporter = CoverageReporter()

# 在 with 代码块中执行被测逻辑
```

```
with tracer:
    run_target()

# 退出 with 后分析执行结果并生成报告
report = reporter.analyze("target.py", tracer)
print(reporter.format_report(report))
```

这段代码的核心只有三步:先用 `trace_file` 指定追踪目标,再在 `with tracer:` 代码块里执行被测程序,最后调用 `CoverageReporter.analyze` 生成覆盖率报告。对于本地函数、测试夹具或小型脚本来说,这种方式已经足够。

4.3.2 本项目中的实际调用方式

第三章实验并不是直接用命令行去跑 `agent_pageindex.py`,而是把 `PathTracer` 嵌入到驱动脚本 `tests/run_agent_pageindex_experiments.py` 中。原因很简单: `agent_pageindex.py` 依赖外部模型和检索工具,若不先固定输入形态,覆盖率结果就难以重复。

代码 4.4 展示了本项目里更接近真实实验的调用方式。

Listing 4.4 本项目中的 `PathTracer` API 用法

```
import contextlib
import io
import runpy
import sys

from pathtracer.reporter import CoverageReporter
from pathtracer.tracer import PathTracer

tracer = PathTracer().trace_file("agent_pageindex.py")
reporter = CoverageReporter()

# 先准备目标脚本的命令行参数
sys.argv = ["agent_pageindex.py", "--query", "test"]

# 在受控实验中,通常会同时做两件事:
# 1. 直接执行真实入口脚本;
# 2. 重定向标准输出,避免目标脚本干扰报告生成。
with tracer, contextlib.redirect_stdout(io.StringIO()):
```

```
runpy.run_path("agent_pageindex.py", run_name="__main__")

report = reporter.analyze("agent_pageindex.py", tracer)
json_text = reporter.format_report_json(report)
```

与最小示例相比, 这里多了两个关键点。第一, 目标脚本是通过 `runpy.run_path(..., run_name="__main__")` 执行的, 因此被追踪对象仍然是真实入口文件; 第二, 实验脚本可以在执行前修改 `sys.argv`、注入桩模块、重定向标准输出, 从而把不稳定的外部因素收束到可控范围内。这也是第三章实验能够稳定复现的原因。

4.3.3 函数调用记录

`PathTracer` 在运行时还会记录目标文件内部的函数调用信息。若需要访问这些数据, 可直接调用 `tracer.get_function_calls()`。这一接口更适合在 `Python API` 场景下使用, 因为当前标准文本报告主要聚焦覆盖率结果, 函数调用记录通常需要调用方自行打印、筛选或二次数列化。

Listing 4.5 读取函数调用记录

```
for call in tracer.get_function_calls():
    print(call.function_name, call.arguments, call.return_value)
```

4.4 报告格式说明

4.4.1 文本报告

文本报告适合直接在终端中查看, 重点展示三个信息: 总控制结构数、已命中控制结构数, 以及按类型统计的分支数量。若不指定 `-o` 参数, `text` 格式会直接输出到标准输出。

Listing 4.6 文本报告示例

```
=====
EXECUTION PATH COVERAGE REPORT
=====

Total Branches: 22
Executed Branches: 16
Coverage: 72.7%
```


Branches by Type:

```
if          : 17
for         : 5
```

```
=====
EXECUTED BRANCHES:
-----
```

agent_pageindex.py:

```
Line 43: for      [executed]
Line 44: if       [executed]
...
```

4.4.2 JSON 报告

JSON 报告适合自动化处理、批量统计或后续实验分析。第三章中的 `docs/coverage_tc1.json`、`docs/coverage_tc2.json` 和 `docs/coverage_tc3.json` 都是由这一格式生成的。

Listing 4.7 JSON 报告结构示例

```
{
  "coverage_percentage": 72.72727272727273,
  "total_branches": 22,
  "executed_branches": 16,
  "branches_by_type": {
    "if": 17,
    "for": 5
  },
  "file_reports": {
    "agent_pageindex.py": {
      "executed_lines": [25, 27, 43, 44, 46],
      "executed_branches": [
        {"line_no": 43, "branch_type": "for", "end_line": 74}
      ]
    }
  },
  "function_calls": []
}
```

```
}
```

这个结构里最重要的部分是 `file_reports`。其中 `executed_lines` 记录动态执行到的代码行，`executed_branches` 记录被静态识别且在本次运行中命中的控制结构入口。对于本课程实验来说，后续统计工作主要也是围绕这两个字段展开。

4.4.3 HTML 报告

HTML 报告适合人工查看和截图展示。报告页面会给出覆盖率摘要、按类型统计的控制结构数量，以及按行着色的源代码视图。已执行行与未执行行采用不同背景色区分，因此更容易定位尚未命中的代码区域。

如果在命令行中选择 `html` 格式但没有指定 `-o`，PathTracer 会默认生成 `coverage-report.html` 文件。对于单文件调试来说，这种方式最直观；对于课程实验统计，则通常仍以 JSON 报告更方便。

4.5 支持范围与结果解释

4.5.1 当前版本支持的控制结构

PathTracer 的静态识别逻辑由 `pathtracer/cfg_builder.py` 实现。当前版本实际统计的控制结构如表 4-2 所示。

表 4-2 当前版本支持的控制结构类型

语法结构	统计类型	说明
<code>if</code> 语句	<code>if</code>	记录条件判断入口行
<code>for</code> 循环	<code>for</code>	记录循环入口行
<code>while</code> 循环	<code>while</code>	记录条件循环入口行
<code>except</code> 处理器	<code>except</code>	记录异常分支入口行

如果这些结构发生嵌套，内部结构仍会被单独统计。例如，一个 `for` 循环内部再包含 `if` 语句，那么这两个入口都会出现在报告中。

需要说明的是，`models.py` 中虽然预留了 `else`、`elif`、`try` 等枚举值，但当前 `CFGBuilder` 还没有把这些结构纳入静态提取结果。因此，在解释当前版本报告时，应以表 4-2 中列出的四类结构为准。

4.5.2 覆盖率指标的含义

本工具当前给出的覆盖率，本质上是**控制结构入口命中率**。也就是说，只要某个 `if`、`for`、`while` 或 `except` 的起始行在运行时被执行到，就会记为“该控制结构已命

中”。

这种统计方式适合用来回答“程序走到了哪些控制结构”这一问题，但它并不等同于严格意义上的真假分支覆盖。例如，某一条 `if` 语句即使只走了真分支，只要条件判断所在行被执行到，该结构也会被记为已覆盖。因此，在阅读 **PathTracer** 报告时，应把它理解为路径分析与测试用例设计的辅助工具，而不是完整的判定覆盖工具。

4.5.3 追踪边界

当前 **PathTracer.trace_file** 的追踪范围限定在目标文件本身。换句话说，若被测脚本在运行时调用了其他模块，这些模块中的代码行不会自动计入当前文件的覆盖率结果。这个边界对本课程实验是可接受的，因为第三章的目标本来就是分析 `agent_pageindex.py` 这一入口脚本的控制流，而不是一次性测透整个 **PageIndex** 项目。

4.6 本项目中的推荐使用流程

结合本项目实际情况，推荐按以下方式使用 **PathTracer**：

- 1) 若只是快速查看某个脚本的覆盖情况，直接使用命令行接口；
- 2) 若目标脚本依赖网络、模型返回值或外部工具结果，则优先使用 **Python API** 封装受控驱动；
- 3) 若需要复现本文第三章的实验结果，则直接运行 `tests/run_agent_pageindex_experiments.py`。

第三章对应的三组实验命令如下：

Listing 4.8 本项目推荐的实验执行命令

```
python3 tests/run_agent_pageindex_experiments.py \
    --scenario normal_string --output docs/coverage_tc1.json

python3 tests/run_agent_pageindex_experiments.py \
    --scenario verbose_mixed --output docs/coverage_tc2.json

python3 tests/run_agent_pageindex_experiments.py \
    --scenario normal_list --output docs/coverage_tc3.json
```

这三个命令并不是 **PathTracer** 的通用 CLI，而是本项目针对 **PageIndex** 入口脚本封装的实验驱动。它们的作用，是在保持被测对象不变的前提下，把外部依赖收束为可控输入，并最终调用 **PathTracer API** 生成统一格式的覆盖率报告。

4.7 本章小结

本章从命令行接口、Python API、报告格式、支持范围和项目内推荐流程五个方面说明了 PathTracer 的实际使用方法。对于一般脚本，命令行接口已经足够；对于像 `agent_pageindex.py` 这样依赖外部组件的真实入口脚本，更合适的做法是通过 Python API 封装受控驱动程序。这样既能保留真实代码路径，又能保证实验结果可重复、可解释。

附录 A 核心源代码

A.1 数据模型 (models.py)

```

1  """PathTracer 数据模型：定义执行路径追踪和覆盖率分析的核心数据结构"""
2
3  from dataclasses import dataclass, field
4  from typing import Set, Dict, List, Optional
5  from enum import Enum
6
7  # 枚举：分支类型，用于标识代码中的控制流结构
8  class BranchType(Enum):
9      SEQUENTIAL = "sequential" # 顺序执行
10     IF = "if" # if 条件分支
11     ELSE = "else" # else 分支
12     ELIF = "elif" # elif 分支
13     FOR = "for" # for 循环
14     WHILE = "while" # while 循环
15     TRY = "try" # try 异常处理块
16     EXCEPT = "except" # except 异常捕获块
17
18 # 数据类：代码分支点
19 # 记录单个分支的位置、类型和执行状态
20 @dataclass
21 class CodeBranch:
22     file_path: str # 源文件路径
23     line_no: int # 分支起始行号
24     branch_type: BranchType # 分支类型
25     end_line: int = 0 # 分支结束行号
26     is_executed: bool = False # 是否被执行过
27
28 # 数据类：函数调用记录
29 # 记录一次函数调用的完整信息，支持嵌套调用树
30 @dataclass
31 class FunctionCall:
32     function_name: str # 函数名
33     file_path: str # 所在文件

```

```

34     line_no: int                                # 调用行号
35     call_depth: int                             # 调用深度 (0=顶层)
36     arguments: Dict[str, str] = field(default_factory=dict) #
参数及值
37     return_value: Optional[str] = None          # 返回值
38     children: List['FunctionCall'] = field(default_factory=list) #
子调用
39
40 # 数据类：执行路径
41 # 记录单个文件的完整执行轨迹
42 @dataclass
43 class ExecutionPath:
44     file_path: str                                # 源文件路径
45     executed_lines: Set[int] = field(default_factory=set) #
已执行行号集合
46     executed_branches: Dict[int, CodeBranch] =
field(default_factory=dict) # 已执行分支
47     function_calls: List[FunctionCall] =
field(default_factory=list) # 函数调用列表
48
49 # 数据类：覆盖率报告
50 # 汇总所有文件的覆盖率统计信息
51 @dataclass
52 class CoverageReport:
53     total_branches: int = 0                        # 总分支数
54     executed_branches: int = 0                    # 已执行分支数
55     branches_by_type: Dict[BranchType, int] =
field(default_factory=dict) # 按类型统计
56     coverage_percentage: float = 0.0             # 覆盖率百分比
57     file_reports: Dict[str, ExecutionPath] =
field(default_factory=dict) # 各文件报告

```

Listing A.1 数据模型定义 (pathtracer/models.py)

A.2 运行时追踪器 (tracer.py)

```

1 """运行时追踪器：利用 Python 的 sys.settrace 机制追踪程序执行路径"""

```

```

2
3 import sys
4 from typing import Optional, Callable, Set, Dict, List
5 from .models import ExecutionPath, FunctionCall
6
7 class PathTracer:
8     """基于 sys.settrace 的 Python 程序执行路径追踪器"""
9
10    def __init__(self):
11        self.execution_paths: Dict[str, ExecutionPath] = {} #
12        各文件的执行路径
13        self._original_trace: Optional[Callable] = None #
14        原始追踪函数 (用于恢复)
15        self._target_file: Optional[str] = None #
16        目标追踪文件
17        self.function_calls: List[FunctionCall] = [] #
18        顶层函数调用列表
19        self._call_stack: List[FunctionCall] = [] #
20        函数调用栈 (用于追踪嵌套)
21
22    def trace_file(self, file_path: str) -> 'PathTracer':
23        """设置要追踪的目标文件, 返回 self 支持链式调用"""
24        self._target_file = file_path
25        self.execution_paths[file_path] =
26        ExecutionPath(file_path=file_path)
27        return self
28
29    def _trace_callback(self, frame, event: str, arg) ->
30        Optional[Callable]:
31        """sys.settrace 的回调函数, 处理各类执行事件"""
32        if not self._target_file:
33            return None
34
35        # 只追踪目标文件中的事件
36        code = frame.f_code
37        if not code.co_filename.endswith(self._target_file):
38            return self._trace_callback

```

```

32
33     # 处理 'line' 事件: 记录执行的代码行
34     if event == 'line':
35         line_no = frame.f_lineno
36
37     self.execution_paths[self._target_file].executed_lines.add(line_no)
38
39     # 处理 'call' 事件: 记录函数调用信息
40     elif event == 'call':
41         func_name = code.co_name
42         line_no = frame.f_lineno
43
44         # 从栈帧中提取函数参数
45         args = {}
46         try:
47             arg_count = code.co_argcount
48             arg_names = code.co_varnames[:arg_count]
49             for arg_name in arg_names:
50                 if arg_name in frame.f_locals:
51                     try:
52                         args[arg_name] =
53 repr(frame.f_locals[arg_name])
54                     except Exception:
55                         args[arg_name] = '<unable to repr>'
56         except Exception:
57             pass
58
59         # 创建 FunctionCall 对象
60         call_depth = len(self._call_stack)
61         func_call = FunctionCall(
62             function_name=func_name,
63             file_path=code.co_filename,
64             line_no=line_no,
65             call_depth=call_depth,
66             arguments=args
67         )

```



```

67         # 将调用添加到父调用的 children 或顶层列表
68         if self._call_stack:
69             self._call_stack[-1].children.append(func_call)
70         else:
71             self.function_calls.append(func_call)
72
73     self.execution_paths[self._target_file].function_calls.append(func_call)
74
75     # 压入调用栈
76     self._call_stack.append(func_call)
77
78     # 处理 'return' 事件: 记录函数返回值
79     elif event == 'return':
80         if self._call_stack:
81             func_call = self._call_stack.pop()
82             if arg is not None:
83                 try:
84                     func_call.return_value = repr(arg)
85                 except Exception:
86                     func_call.return_value = '<unable to repr>'
87
88     return self._trace_callback
89
90 def __enter__(self):
91     """上下文管理器入口: 启动追踪"""
92     self._original_trace = sys.gettrace()
93     sys.settrace(self._trace_callback)
94     return self
95
96 def __exit__(self, exc_type, exc_val, exc_tb):
97     """上下文管理器出口: 停止追踪"""
98     sys.settrace(self._original_trace)
99     return False
100
101 def get_executed_lines(self, file_path: str) -> Set[int]:
102     """获取指定文件的已执行行号集合"""
103     if file_path in self.execution_paths:

```

```

103         return self.execution_paths[file_path].executed_lines
104     return set()
105
106     def get_function_calls(self) -> List[FunctionCall]:
107         """获取记录的所有函数调用"""
108         return self.function_calls

```

Listing A.2 运行时追踪器实现 (pathtracer/tracer.py)

A.3 控制流图构建器 (cfg_builder.py)

```

1  """控制流图构建器：通过静态分析 AST 提取代码中的所有分支点"""
2
3  import ast
4  from typing import List, Optional
5  from .models import CodeBranch, BranchType
6
7  class CFGBuilder:
8      """从 Python 源代码构建控制流图，识别所有分支结构"""
9
10     def __init__(self):
11         self.branches: List[CodeBranch] = [] # 提取的分支列表
12
13     def build_from_file(self, file_path: str) -> List[CodeBranch]:
14         """解析 Python 文件并提取所有分支点"""
15         with open(file_path, 'r', encoding='utf-8') as f:
16             source = f.read()
17
18         # 解析源代码为 AST
19         tree = ast.parse(source, filename=file_path)
20         self.branches = []
21
22         # 遍历 AST 查找控制流结构
23         for node in ast.walk(tree):
24             branch = self._extract_branch(file_path, node)
25             if branch:
26                 self.branches.append(branch)

```

```
27
28     return self.branches
29
30     def _extract_branch(self, file_path: str, node: ast.AST) ->
Optional[CodeBranch]:
31         """从 AST 节点提取分支信息"""
32
33         # if 语句
34         if isinstance(node, ast.If):
35             return CodeBranch(
36                 file_path=file_path,
37                 line_no=node.lineno,
38                 branch_type=BranchType.IF,
39                 end_line=self._get_end_line(node)
40             )
41
42         # for 循环
43         elif isinstance(node, ast.For):
44             return CodeBranch(
45                 file_path=file_path,
46                 line_no=node.lineno,
47                 branch_type=BranchType.FOR,
48                 end_line=self._get_end_line(node)
49             )
50
51         # while 循环
52         elif isinstance(node, ast.While):
53             return CodeBranch(
54                 file_path=file_path,
55                 line_no=node.lineno,
56                 branch_type=BranchType.WHILE,
57                 end_line=self._get_end_line(node)
58             )
59
60         # try/except 异常处理
61         elif isinstance(node, ast.ExceptHandler):
62             return CodeBranch(
```

```

63         file_path=file_path,
64         line_no=node.lineno,
65         branch_type=BranchType.EXCEPT,
66         end_line=self._get_end_line(node)
67     )
68
69     return None
70
71     def _get_end_line(self, node: ast.AST) -> int:
72         """获取代码块的最后一行行号"""
73         # Python 3.8+ 支持 end_lineno 属性
74         if hasattr(node, 'end_lineno') and node.end_lineno is not
None:
75             return node.end_lineno
76         # 递归查找 body 中最大的行号
77         if hasattr(node, 'body') and node.body:
78             return max(self._get_end_line(n) for n in node.body)
79         return getattr(node, 'lineno', 0)
80
81     def get_branches_by_type(self, branch_type: BranchType) ->
List[CodeBranch]:
82         """按类型筛选分支"""
83         return [b for b in self.branches if b.branch_type ==
branch_type]

```

Listing A.3 控制流图构建器实现 (pathtracer/cfg_builder.py)

A.4 报告生成器 (reporter.py)

```

1  """覆盖率报告生成器：对比运行时追踪结果与静态 CFG 分析，生成覆盖率报告"""
2
3  import json
4  from typing import Dict, Any, List
5  from .models import CodeBranch, ExecutionPath, CoverageReport,
    BranchType, FunctionCall
6  from .cfg_builder import CFGBuilder
7  from .tracer import PathTracer

```

```
8
9 class CoverageReporter:
10     """生成覆盖率报告：比较追踪路径与控制流图"""
11
12     def __init__(self):
13         self.cfg_builder = CFGBuilder() # 用于静态分析
14
15     def analyze(self, file_path: str, tracer: PathTracer) ->
CoverageReport:
16         """分析指定文件的覆盖率"""
17         # 静态分析：获取所有分支
18         all_branches = self.cfg_builder.build_from_file(file_path)
19
20         # 运行时数据：获取已执行的行
21         executed_lines = tracer.get_executed_lines(file_path)
22
23         # 标记已执行的分支
24         for branch in all_branches:
25             if branch.line_no in executed_lines:
26                 branch.is_executed = True
27
28         # 计算统计数据
29         total = len(all_branches)
30         executed = sum(1 for b in all_branches if b.is_executed)
31
32         # 按分支类型统计
33         by_type: Dict[BranchType, int] = {}
34         for branch_type in BranchType:
35             type_branches = [b for b in all_branches if
b.branch_type == branch_type]
36             by_type[branch_type] = len(type_branches)
37
38         # 构建并返回覆盖率报告
39         return CoverageReport(
40             total_branches=total,
41             executed_branches=executed,
42             branches_by_type=by_type,
```

```

43         coverage_percentage=(executed / total * 100) if total >
0 else 0.0,
44         file_reports={file_path: ExecutionPath(
45             file_path=file_path,
46             executed_lines=executed_lines,
47             executed_branches={b.line_no: b for b in
all_branches if b.is_executed}
48         )}
49     )
50
51     def format_report(self, report: CoverageReport) -> str:
52         """将覆盖率报告格式化为可读文本"""
53         lines = [
54             "=" * 60,
55             "EXECUTION PATH COVERAGE REPORT",
56             "=" * 60,
57             "",
58             f"Total Branches: {report.total_branches}",
59             f"Executed Branches: {report.executed_branches}",
60             f"Coverage: {report.coverage_percentage:.1f}%",
61             "",
62             "Branches by Type:",
63         ]
64
65         # 按类型输出分支统计
66         for branch_type, count in report.branches_by_type.items():
67             if count > 0:
68                 lines.append(f"    {branch_type.value:12s}: {count}")
69
70         lines.extend([
71             "",
72             "=" * 60,
73             "EXECUTED BRANCHES:",
74             "-" * 60,
75         ])
76
77         # 输出各文件的已执行分支详情

```

```

78         for file_path, exec_path in report.file_reports.items():
79             lines.append(f"\n{file_path}:")
80             for line_no, branch in
sorted(exec_path.executed_branches.items()):
81                 lines.append(
82                     f"   Line {branch.line_no:4d}:
{branch.branch_type.value:10s} [executed]"
83                     )
84
85         return "\n".join(lines)
86
87     def format_report_json(self, report: CoverageReport) -> str:
88         """将覆盖率报告格式化为 JSON (便于程序处理)"""
89         data: Dict[str, Any] = {
90             "coverage_percentage": report.coverage_percentage,
91             "total_branches": report.total_branches,
92             "executed_branches": report.executed_branches,
93             "branches_by_type": {
94                 bt.value: count for bt, count in
report.branches_by_type.items()
95             },
96             "file_reports": {},
97             "function_calls": []
98         }
99
100         # 构建各文件的报告数据
101         for file_path, exec_path in report.file_reports.items():
102             file_data: Dict[str, Any] = {
103                 "executed_lines":
sorted(list(exec_path.executed_lines)),
104                 "executed_branches": []
105             }
106             for line_no, branch in
sorted(exec_path.executed_branches.items()):
107                 file_data["executed_branches"].append({
108                     "line_no": branch.line_no,
109                     "branch_type": branch.branch_type.value,

```

```

110         "end_line": branch.end_line
111     })
112     data["file_reports"][file_path] = file_data
113
114     # 包含函数调用信息
115     if exec_path.function_calls:
116         data["function_calls"] =
117         self._serialize_function_calls(
118             exec_path.function_calls
119         )
120
121     return json.dumps(data, indent=2)
122
123     def _serialize_function_calls(self, calls: List[FunctionCall])
124     -> List[Dict[str, Any]]:
125         """递归序列化函数调用树为 JSON 可用的字典列表"""
126         result = []
127         for call in calls:
128             call_dict = {
129                 "function_name": call.function_name,
130                 "file_path": call.file_path,
131                 "line_no": call.line_no,
132                 "call_depth": call.call_depth,
133                 "arguments": call.arguments,
134                 "return_value": call.return_value,
135                 "children":
136                 self._serialize_function_calls(call.children) # 递归处理子调用
137             }
138             result.append(call_dict)
139         return result

```

Listing A.4 报告生成器实现 (pathtracer/reporter.py)

A.5 命令行接口 (cli.py)

```

1  """PathTracer 命令行接口：提供用户友好的命令行交互方式"""
2

```



```
3 import sys
4 import argparse
5 import io
6 from pathlib import Path
7 from .tracer import PathTracer
8 from .reporter import CoverageReporter
9
10 def main():
11     """CLI 主入口函数"""
12     raw_argv = sys.argv[1:]
13
14     # 无参数时显示帮助信息
15     if not raw_argv:
16         parser = argparse.ArgumentParser(
17             description='Trace Python program execution paths and
generate coverage reports'
18         )
19         parser.add_argument('program', help='Python program file to
trace')
20         parser.add_argument('--output', '-o', help='Output file for
report', default=None)
21         parser.add_argument('--quiet', '-q', help='Only output
report', action='store_true')
22         parser.add_argument(
23             '--format', '-f',
24             choices=['text', 'html', 'json'],
25             default='text',
26             help='Output format: text, html, or json (default:
text)'
27         )
28         parser.add_argument('program_args', nargs='*',
help='Arguments for target program')
29         parser.parse_args(raw_argv)
30         return
31
32     # 解析 CLI 参数与目标程序参数 (以第一个位置参数为分界)
33     cli_argv = []
```

```

34     program_args = []
35     program = None
36     index = 0
37     while index < len(raw_argv):
38         arg = raw_argv[index]
39         # 处理带值的选项
40         if arg in ('--output', '-o', '--format', '-f'):
41             if index + 1 >= len(raw_argv):
42                 cli_argv.append(arg)
43                 break
44             cli_argv.extend([arg, raw_argv[index + 1]])
45             index += 2
46             continue
47         # 处理标志选项
48         if arg in ('--quiet', '-q'):
49             cli_argv.append(arg)
50             index += 1
51             continue
52         # 第个位置参数是目标程序, 后续为其参数
53         program = arg
54         cli_argv.append(arg)
55         program_args = raw_argv[index + 1:]
56         break
57
58     # 构建参数解析器
59     parser = argparse.ArgumentParser(
60         description='Trace Python program execution paths and
generate coverage reports'
61     )
62     parser.add_argument('program', help='Python program file to
trace')
63     parser.add_argument('--output', '-o', help='Output file for
report', default=None)
64     parser.add_argument('--quiet', '-q', help='Only output report',
action='store_true')
65     parser.add_argument(
66         '--format', '-f',

```

```
67     choices=['text', 'html', 'json'],
68     default='text',
69     help='Output format: text, html, or json (default: text)'
70 )
71 parser.add_argument('program_args', nargs='*', help='Arguments
for target program')
72
73 args = parser.parse_args(cli_argv)
74
75 # 检查目标文件是否存在
76 if not Path(args.program).exists():
77     print(f"Error: File not found: {args.program}",
file=sys.stderr)
78     sys.exit(1)
79
80 # 处理参数分隔符 --
81 if program_args[:1] == ['--']:
82     program_args = program_args[1:]
83
84 # 创建追踪器并设置目标文件
85 tracer = PathTracer().trace_file(args.program)
86
87 # 静默模式：捕获目标程序的 stdout
88 if args.quiet:
89     old_stdout = sys.stdout
90     sys.stdout = io.StringIO()
91
92 old_argv = sys.argv
93 reporter = CoverageReporter()
94
95 try:
96     # 设置 sys.argv 以便目标程序获取正确参数
97     sys.argv = [args.program] + program_args
98     # 在追踪上下文中执行目标程序
99     with tracer:
100         with open(args.program) as f:
101             code = compile(f.read(), args.program, 'exec')
```

```

102         exec(code, {'__name__': '__main__', '__file__':
args.program})
103     finally:
104         # 恢复 sys.argv 和 stdout
105         sys.argv = old_argv
106         if args.quiet:
107             sys.stdout = old_stdout
108
109         # 生成覆盖率报告
110         report = reporter.analyze(args.program, tracer)
111
112         # 根据格式选择输出方法
113         if args.format == 'json':
114             output = reporter.format_report_json(report)
115         elif args.format == 'html':
116             output = reporter.format_report_html(report)
117         else:
118             output = reporter.format_report(report)
119
120         # 确定输出文件
121         output_file = args.output
122         if not output_file:
123             if args.format == 'html':
124                 output_file = 'coverage-report.html'
125             elif args.format == 'json':
126                 output_file = 'coverage-report.json'
127
128         # 输出报告
129         if output_file:
130             with open(output_file, 'w') as f:
131                 f.write(output)
132             print(f"Report written to: {output_file}")
133         else:
134             print(output)
135
136 if __name__ == '__main__':
137     main()

```

Listing A.5 命令行接口实现 (pathtracer/cli.py)